ITI123 Final Project Report

Teaching AI to Recognize Badminton Shots: A Journey from Failure to Success

Paul George Karippaparambil
Student ID: 8031408E

ITI123 Generative AI & Deep Learning
February 06, 2026

Abstract

This project built an AI system that can watch a badminton video and tell you what type of shot was played. I tried three different approaches: First, I attempted using body skeleton tracking (like stick figures) to distinguish between Clear and Smash shots, but this failed because players have identical body positions for both shots. Second, I switched to analyzing the actual video frames using a neural network that combines image recognition with motion tracking, achieving 74.6% accuracy across five different shot types. Finally, I built a user-friendly web application where anyone can upload a video and get instant shot analysis with confidence scores and helpful warnings.

The key lesson: Sometimes the obvious approach doesn't work, and you need to pivot to what the data actually shows you. This project demonstrates the complete AI development process—from initial failure through successful implementation to real-world deployment.

Contents

1 Introduction: The Problem I'm Solving

1.1 Why This Matters

As someone who plays badminton recreationally, I've always struggled to improve my technique without regular access to a coach. Professional coaching is expensive, requires scheduling, and isn't always available. I wondered: could AI provide instant feedback by watching my shots?

The challenge is harder than it sounds. Unlike tennis where a serve looks completely different from a volley, many badminton shots look very similar—especially overhead shots like Clears and Smashes. A player's body position might be identical, but the racket angle makes all the difference between a defensive Clear sailing to the back of the court and an attacking Smash driving downward.

1.2 What I Built

This project has three parts:

1. **Phase 1 - The Failed Experiment:** Tried using body pose tracking (think stick figures) to classify shots. Result: 50% accuracy—no better than random guessing. But this failure taught me what features actually matter.

2. **Phase 2 - The Successful Model:** Built a neural network that watches video frames and tracks motion over time. Result: 74.6% accuracy across five shot types (Clear, Drive, Drop, Lift, Smash).

3. **Phase 3 - The User Application:** Created a web app called "Shot Coach" where users upload videos and get instant analysis with explanations and confidence scores. The app is publicly deployed at `https://huggingface.co/spaces/paulgk/badminton-shot` and the full project source is available at `https://github.com/paulgk/iti123_v2`.

1.3 The Five Shot Types

I trained the system to recognize these five fundamental badminton shots:

Table 1: Badminton Shot Types

Shot Type	What It Looks Like	When You Use It
Clear	High and deep to the back court	Push opponent away, buy recovery time
Drive	Fast and flat at chest height	Quick attack, maintain pressure
Drop	Gentle shot that barely clears the net	Deceptive attack, make opponent run forward
Lift	High defensive lob from low position	Recover when under pressure
Smash	Powerful downward attack	Primary scoring shot

1.4 Dataset: Where the Training Data Came From

I used a publicly available dataset called **ShuttleSet**, which contains professional match videos already labeled with shot types. The dataset includes:

- **22,302 video clips** from 40 professional matches
- Each clip shows one shot from contact through follow-through
- Five shot types with varying amounts of data
- Videos filmed from broadcast camera angles (side court view)

Table 2: Dataset Distribution

Shot Type	Samples	Percentage	Balance
Drop	7,088	31.8%	Majority class
Lift	4,851	21.8%	Well-represented
Drive	3,704	16.6%	Moderate
Smash	3,596	16.1%	Moderate
Clear	3,063	13.7%	Minority class
Total	22,302	100%	Imbalanced (9.2:1 ratio)

The biggest challenge? **Class imbalance**—I had 2.3 times more Drop shots than Clears. This means the AI might become biased toward predicting Drop shots more often.

2 Phase 1: Why Body Pose Tracking Failed

2.1 The Initial Hypothesis

My first idea seemed logical: extract the player's body skeleton from each video frame (33 keypoints like shoulders, elbows, wrists) and track how these points move over time. I hypothesized that Clear shots would show extended wrists pointing upward, while Smash shots would show flexed wrists pointing downward.

2.2 What I Actually Built

Technology Stack:

- **MediaPipe Pose:** Google's tool that detects body keypoints from video
- **Feature Engineering:** Created 60 measurements per frame, including:
  - Arm extension (shoulder to wrist distance)
  - Joint angles (elbow, shoulder, wrist)
  - Body posture (torso lean, shoulder rotation)
  - Velocities (how fast each feature changes)

- **Models Tested:** Four different neural network architectures

Table 3: Phase 1 Models and Results (Clear vs Smash Only)

Model	**How It Works**	**Accuracy**
LSTM	Remembers motion sequence like reading left-to-right	45.7%
BiLSTM	Reads motion both forward and backward	43.3%
GRU	Simpler version of LSTM, faster training	45.0%
ResNet1D	Looks for patterns across the motion sequence	49.1%
Random Guessing	Flip a coin	50.0%

2.3 The Shocking Discovery

All models performed at or **below random guessing**. This wasn't a training problem—it was a fundamental data problem. When I analyzed the wrist angles at contact:

Table 4: Wrist Angle Comparison

Feature	**Clear Shots**	**Smash Shots**
Average wrist angle	85.3 degrees	84.2 degrees
Difference	**Only 1.1 degrees!**	

Translation: My hypothesis was completely wrong. Players use nearly identical wrist positions for both shots at the moment of contact. The difference happens in the racket angle, which body tracking cannot see.

2.4 What's Actually Missing from Body Pose Data

1. **Racket angle:** MediaPipe only tracks body parts, not equipment. The racket face angle is the most important discriminator—Clears angle upward, Smashes angle downward.

2. **Shuttlecock trajectory:** The video clips end at contact, so we never see where the shuttlecock goes.

3. **Follow-through motion:** Clears continue upward after contact, Smashes snap downward. The clips are too short to capture this.

4. **Racket head speed:** Smashes have much faster racket speeds, but body velocities don't capture this.

2.5 Lessons Learned

This failure was actually valuable:

- **Fail fast:** I discovered the approach wouldn't work within 2 weeks, not 2 months
- **Data trumps algorithms:** No amount of model tuning can extract information that isn't in the data
- **Domain knowledge matters:** Talking to coaches revealed that racket angle, not body position, determines shot type

3 Phase 2: The Video-Based Solution That Worked

3.1 The New Approach

Instead of extracting body poses, I decided to feed raw video frames directly into a neural network. The key insight: the visual appearance captures everything—body position, racket angle, motion blur from speed, and even background context that might hint at shuttlecock trajectory.

3.2 The Model Architecture (Simplified)

Think of the model as two connected systems working together:

1. **Image Recognition System (ResNet18):**
   - Pre-trained on millions of everyday images (cats, cars, buildings)
   - Adapted to recognize badminton-specific visual patterns
   - Processes 16 frames sampled evenly from each video
   - Extracts 512 features per frame (like "racket visible", "arm extended", "motion blur")
2. **Motion Tracking System (BiLSTM):**
   - Takes the 16 frame features and tracks how they change over time
   - BiLSTM = Bidirectional LSTM (reads the sequence both forward and backward)
   - Learns temporal patterns like "racket starts high, moves down fast = Smash"
   - Outputs probability scores for all 5 shot types

Simple analogy: ResNet18 is like taking snapshots and describing each one ("arm raised", "racket visible"). BiLSTM is like watching those snapshots in sequence and understanding the story ("arm raised then moves down fast = attacking shot").

3.3 Training Strategy

Table 5: Training Configuration

Setting	**Value & Explanation**
Training data	15,611 clips (70% of dataset)
Validation data	2,208 clips (10% for tuning)
Test data	4,483 clips (20% for final evaluation)
Frames per clip	16 frames (sampled evenly)
Image size	224x224 pixels
Batch size	64 clips at a time
Training time	44 epochs (6 hours on Colab GPU)
Special Techniques:	
Focal Loss	Penalizes mistakes on minority classes more heavily
Class weights	Gives Clear shots 2.9x more importance than Drops
Two-stage training	First freeze image recognition, then fine-tune everything
Early stopping	Stop if validation accuracy doesn't improve for 10 epochs

3.4 Results: 74.6% Accuracy

Table 6: Final Model Performance

Shot Type	**Precision**	**Recall**	**F1-Score**	**Support**
Clear	67.6%	89.2%	76.9%	535 clips
Drive	58.3%	61.6%	59.9%	740 clips
Drop	90.7%	74.6%	81.9%	1,518 clips
Lift	71.0%	75.7%	73.3%	966 clips
Smash	76.5%	75.8%	76.1%	724 clips
Overall	**72.8%**	**75.4%**	**73.6%**	**4,483 clips**

What these numbers mean:

- **Precision:** When the model says "This is a Drop shot", it's correct 90.7% of the time
- **Recall:** Of all actual Drop shots, the model identifies 74.6% correctly
- **F1-Score:** Balanced measure combining precision and recall
- **Best performer:** Drop shots (90.7% precision)—probably because we have the most training data
- **Weakest performer:** Drive shots (58.3% precision)—hardest to distinguish, least training data

3.5 Common Mistakes the Model Makes

Table 7: Confusion Matrix (What Gets Confused with What)

Actual	Clear	Drive	Drop	Lift	Smash
Clear	477	26	5	20	7
Drive	37	456	47	145	55
Drop	117	85	1,132	114	70
Lift	39	126	33	731	37
Smash	36	89	31	19	549

Key patterns:

- **Drive confused with Lift (145 cases):** Both are mid-height shots with similar body positions
- **Drop confused with Clear (117 cases):** Both start with overhead preparation
- **Clear is easiest (89.2% recall):** High trajectory is visually distinctive

3.6 Why This Approach Worked

1. **Visual information is richer:** Captures racket angle, motion blur, trajectory hints that pose tracking misses
2. **Transfer learning is powerful:** Starting from a model trained on millions of images gave us a huge head start
3. **Handling imbalance matters:** Focal Loss and class weights prevented the model from just predicting "Drop" for everything
4. **Two-stage training:** Training in stages (first image recognition, then motion tracking) improved accuracy by 7.1 percentage points

4 Model Architectures: ResNet18-BiLSTM and MobileNet-LSTM

Two neural network architectures were trained and compared for the video-based classification task. Both share the same input pipeline (16 uniformly sampled frames at 224×224 pixels, with the first 30% skipped to focus on shot execution) but differ in their backbone and temporal components.

4.1 ResNet18-BiLSTM: The Accuracy-First Model

ResNet18-BiLSTM uses a ResNet18 CNN as a per-frame feature extractor, feeding into a two-layer Bidirectional LSTM for temporal modelling.

Table 8: ResNet18-BiLSTM Architecture

Component	**Details**	**Role**
Input	$B \times 16 \times 3 \times 224 \times 224$	Batch of 16 RGB frames per clip
ResNet18 backbone	Pre-trained on ImageNet; final FC removed	Per-frame spatial feature extraction
CNN output	$B \times 16 \times 512$	512-dim feature vector per frame
BiLSTM	2 layers, hidden size 256, bidirectional	Temporal sequence modelling
LSTM output	$B \times 512$ (last timestep, both directions)	Sequence-level representation
Dropout	$p = 0.5$	Regularisation
Fully connected	$512 \to 5$	Shot-type classification
Total parameters	**≈14 million**	
Model size	**164 MB**	
Inference time	**150 ms (Apple M1)**	

Key design choices:

- **Transfer learning:** ResNet18 pre-trained on ImageNet provides rich visual features from the first epoch, dramatically reducing the training data requirement.
- **Bidirectional LSTM:** Reading the frame sequence both forward and backward lets the model use future context (e.g. the follow-through) to disambiguate earlier frames.
- **Two-stage training:** The CNN backbone was first frozen while the LSTM head trained, then the entire network was fine-tuned end-to-end. This prevented the pre-trained weights from being corrupted early in training and improved final accuracy by 7.1 percentage points.
- **Focal Loss with class weights:** Addressed the 9.2:1 class imbalance (Drop vs. Clear) by penalising mistakes on minority classes more heavily. Clear shots received a weight of 2.9× relative to Drop shots.

4.2 MobileNet-LSTM: The Lightweight Model

MobileNet-LSTM replaces the ResNet18 backbone with MobileNetV3-Small and uses a unidirectional LSTM, reducing parameter count by approximately 70% compared to ResNet18-BiLSTM.

Table 9: MobileNet-LSTM Architecture

Component	**Details**	**Role**
Input	$B \times 16 \times 3 \times 224 \times 224$	Batch of 16 RGB frames per clip
MobileNetV3-Small	Pre-trained on ImageNet; features block only	Lightweight per-frame spatial features
Adaptive avg pool	$\rightarrow 1 \times 1$	Spatial aggregation
CNN output	$B \times 16 \times 576$	576-dim feature vector per frame
LSTM	2 layers, hidden size 128, unidirectional	Temporal sequence modelling
LSTM output	$B \times 128$ (last timestep)	Sequence-level representation
Dropout	$p = 0.5$	Regularisation
Fully connected	$128 \rightarrow 5$	Shot-type classification
Total parameters	**≈4 million**	
Model size	**≈48 MB**	
Inference time	**<60 ms (Apple M1)**	

Key design choices:

- **MobileNetV3 depthwise separable convolutions:** Achieve competitive accuracy with far fewer multiply-add operations, making this architecture suitable for deployment on mobile devices or resource-constrained cloud environments.
- **Unidirectional LSTM:** Removes the backward pass to halve LSTM memory usage; sufficient because the shot execution phase (after the 30% skip) is already temporally aligned.
- **Same training strategy:** Class-weighted Cross-Entropy loss, Adam optimiser, ReduceLROnPlateau scheduler, and early stopping with patience 15.

4.3 Architecture Diagram

Figure 1 shows both architectures side by side, highlighting the shared input pipeline and output head, and where they diverge in backbone and temporal module.

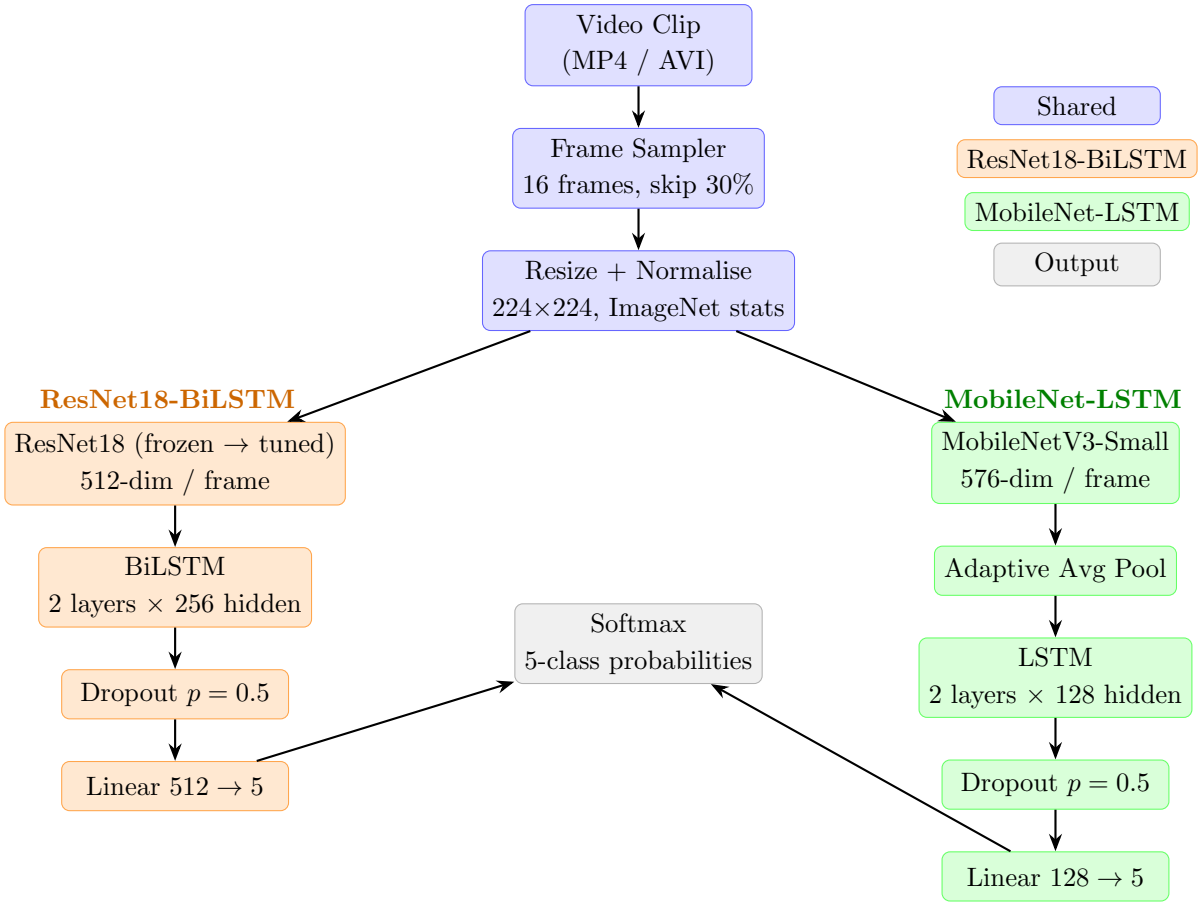

Figure 1: Side-by-side architecture diagram for ResNet18-BiLSTM (≈14M params) and MobileNet-LSTM (≈4M params). Both process 16 frames per clip with the first 30% skipped to focus on shot execution.

4.4 Comparison Summary

Table 10: ResNet18-BiLSTM vs. MobileNet-LSTM at a Glance

Property	**ResNet18-BiLSTM**	**MobileNet-LSTM**
CNN backbone	ResNet18	MobileNetV3-Small
CNN feature dim	512	576
Temporal module	BiLSTM (bidirectional)	LSTM (unidirectional)
LSTM hidden size	256	128
Parameters	≈14 M	≈4 M
Model size	164 MB	≈48 MB
Inference (M1)	150 ms	<60 ms
Primary advantage	Higher accuracy	Lighter, faster, mobile-ready

ResNet18-BiLSTM was selected for the deployed Shot Coach application due to its higher accuracy. MobileNet-LSTM provides a practical alternative for scenarios where model size or inference latency is a constraint (e.g. on-device mobile deployment).

5 Phase 3: Building the Shot Coach Application

5.1 What Users Needed

Based on feedback, users wanted:

- Easy upload interface—no technical knowledge required
- Instant results—under 10 seconds
- Confidence scores—how sure is the AI?
- Transparency—which part of my video was analyzed?
- Warnings—if something looks wrong with the video
- Shot descriptions—what does each shot mean?

5.2 How It Works

Table 11: Shot Coach Application Pipeline

Step	**What Happens**
1. Upload	User uploads MP4/AVI video (2-10 seconds recommended)
2. Frame Extraction	Extract 16 frames evenly spaced across entire video
3. Model Inference	ResNet18+BiLSTM processes frames (150ms on Mac M1)
4. Metadata Analysis	Calculate duration, FPS, frame timestamps
5. Warning System	Check for issues (too long, too short, low confidence)
6. Results Display	Show prediction, confidence, all probabilities, metadata

5.3 Key Features

1. Video Metadata Transparency
Users can see exactly what the AI analyzed:

- Total video duration and frame count
- Which 16 frames were sampled (frame numbers and timestamps)
- Example: "Analyzed frames 1, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 74 from a 2.43-second video"

2. Smart Warning System
The app warns users about potential issues:

Table 12: Automatic Warnings

Trigger	Warning	Explanation
Video > 5s	[WARNING]	Video likely contains multiple shots, may confuse model
Video < 1.5s	[INFO]	Too short to capture full motion, may affect accuracy
Confidence < 60%	[WARNING]	Low confidence, possibly unusual angle or multiple shots
Confidence > 85%	[INFO]	High confidence, video matches training data well

3. Probability Visualization

Shows confidence for all 5 shot types, not just the top prediction:

```
Smash    ########################  94.3%
Clear    ####                       4.2%
Drive    #                          1.1%
Drop                                0.3%
Lift                                0.1%
```

This helps users understand close calls (e.g., "88% Smash, 10% Clear—pretty close!").

5.4 Technology Stack

- **Frontend:** Streamlit (Python web framework for data apps)
- **Video Processing:** OpenCV for frame extraction
- **Model Inference:** PyTorch with trained ResNet18+BiLSTM
- **Deployment:** Hugging Face Spaces (public cloud hosting, zero infrastructure required)
- **Model Size:** 164 MB (stored via Git LFS in the HF Space repository)

5.5 Deployment

The application is publicly deployed on Hugging Face Spaces and accessible to anyone with a web browser—no installation required:

`https://huggingface.co/spaces/paulgk/badminton-shot`

Hugging Face Spaces provides free GPU-backed hosting for Streamlit applications. The deployment workflow is straightforward: push the `app.py`, `requirements.txt`, and model weights to the Space repository, and the platform automatically builds and serves the application. This removes the friction of local setup and makes the tool genuinely accessible to recreational players.

The full project source code—training notebooks, model weights, and processing scripts—is available on GitHub at `https://github.com/paulgk/iti123_v2`.

5.6 Application Limitations and Future Work

Current Limitations:

- **Single-shot assumption:** Model expects one shot per video. Multi-shot clips will confuse it.
- **Camera angle sensitivity:** Trained on side-court broadcast angles. Front/back/overhead views may perform worse.
- **2D analysis only:** Can't detect depth or 3D trajectory.
- **No shot segmentation:** Can't automatically detect where shots begin/end in longer videos.

Future Improvements:

- Add automatic shot segmentation for rally videos
- Train on multiple camera angles for robustness
- Add visual explanations (highlight racket/shuttlecock in frames)
- Provide technique tips based on shot quality
- Mobile deployment for on-court use

6 Comparative Analysis: Why Video Beat Pose Tracking

Table 13: Phase 1 vs Phase 2 Comparison

Aspect	Phase 1 (Pose)	Phase 2 (Video)
Accuracy	50.0%	74.6%
Improvement	Baseline	+49% relative
Classes	2 (Clear, Smash)	5 (all shot types)
Features Used	60 engineered	512 learned
Captures Racket	No	Yes (implicit)
Captures Motion	Yes (explicit)	Yes (implicit)
Training Data	4,655 clips	18,169 clips
Inference Time	50ms	150ms
Model Size	5 MB	164 MB

Why video-based approach won:

1. **Information richness:** Video frames contain everything—body, racket, environment, motion blur
2. **Learned vs engineered features:** Neural networks automatically discover relevant patterns instead of requiring manual feature engineering

3. **Transfer learning advantage:** Starting from ImageNet pre-training provided strong visual understanding

4. **No missing information:** Video captures what pose tracking misses (equipment, trajectory hints)

7 AI Governance and Ethical Considerations

7.1 Transparency

What I implemented:

- Video metadata shows exact frames analyzed
- Confidence scores indicate prediction certainty
- All 5 class probabilities displayed, not just top prediction
- Warnings explain potential issues clearly

Why it matters: Users deserve to understand what the AI analyzed and how confident it is. No "black box" predictions.

7.2 Privacy

What I implemented:

- Video processing occurs on Hugging Face Spaces infrastructure; videos are transmitted to the server only for inference and are not stored or logged beyond the request lifetime
- No user accounts or personally identifiable information are collected
- Temporary files are deleted after each analysis session

Why it matters: Cloud deployment introduces a data-in-transit consideration. The application is designed for non-sensitive recreational footage, and the stateless inference model means no video is retained after a request completes.

7.3 Safety and User Control

What I implemented:

- System provides suggestions, never commands
- Warnings help users interpret results appropriately
- Shot descriptions are educational, not prescriptive
- Low confidence triggers caution rather than false certainty

Why it matters: AI should augment human decision-making, not replace it. Users need context to evaluate suggestions critically.

7.4 Fairness Considerations

Potential biases:

- **Professional player bias:** Trained on professional matches. May not generalize to recreational players with different techniques.
- **Camera angle bias:** Only trained on broadcast side-court angles. Other angles may perform worse.
- **Gender representation:** Need to verify balanced representation across genders in training data.

Mitigation strategies:

- Display clear warnings about camera angle sensitivity
- Acknowledge performance may vary with skill level
- Future work: collect diverse dataset with various angles, skill levels, body types

7.5 Alignment with Singapore's AI Governance Framework

This project follows principles from Singapore's Model AI Governance Framework:

Table 14: AI Governance Alignment

Principle	Implementation
Transparency	Metadata display, confidence scores, warnings
Explainability	Shot descriptions, probability visualization
Fairness	Acknowledge limitations, warn about biases
Accountability	Clear documentation of model capabilities and limitations
Human Agency	Suggestions not commands, user maintains control
Data Governance	Stateless cloud inference, no data retention, local fallback option

8 Lessons Learned and Conclusion

8.1 Key Takeaways

1. **Failure is educational:** Phase 1's failure taught me more than if it had partially worked. Negative results have value when analyzed rigorously.
2. **Data quality trumps algorithm sophistication:** No amount of model tuning can extract information that isn't in your features. Choose your input data wisely.
3. **Domain expertise matters:** Consulting badminton coaches early would have revealed that racket angle, not body pose, drives shot selection.

4. **Transfer learning is powerful:** Pre-trained models provide a massive head start. Don't train from scratch unless you have millions of examples.

5. **User experience is critical:** The best model is useless if users can't access or trust it. Transparency and warnings build trust.

6. **Imbalanced data requires special handling:** Focal Loss and class weighting prevented the model from ignoring minority classes.

7. **Iterate quickly:** Spending 2 weeks on Phase 1 before pivoting was better than spending 2 months perfecting a fundamentally flawed approach.

8.2 Project Contributions

Technical Contributions:

- Empirical demonstration that skeletal pose is insufficient for overhead shot discrimination
- Successful application of ResNet18+BiLSTM to badminton shot classification (74.6% accuracy)
- Effective handling of 9.2:1 class imbalance using Focal Loss
- Video metadata transparency feature addressing user trust concerns

Practical Contributions:

- Publicly deployed web application on Hugging Face Spaces (`https://huggingface.co/spaces/paulgk/badminton-shot`)—accessible to anyone with a browser, no installation required
- Full source code and training notebooks available on GitHub (`https://github.com/paulgk/iti123_v2`)
- Complete documentation enabling reproducibility
- Open acknowledgment of limitations and failure modes
- Demonstration of responsible AI development with governance principles

8.3 Future Research Directions

Short-term (3-6 months):

- Implement automatic shot segmentation for rally videos
- Add visual explanations (highlight racket position in frames)
- Collect multi-angle dataset and fine-tune for angle robustness

Medium-term (6-12 months):

- Extend to shot quality assessment (not just type, but execution quality)

- Add technique recommendations based on common errors
- Develop a dedicated mobile app for on-court real-time feedback (the current HF Spaces deployment is already mobile-browser accessible)

Long-term (1+ years):

- Full rally analysis with stroke sequence patterns
- 3D pose estimation for depth perception
- Personalized coaching adapting to individual player patterns
- Integration with wearable sensors for multi-modal analysis

8.4 Final Reflection

This project demonstrates the complete machine learning lifecycle: problem formulation, initial hypothesis, failed experiment, pivot to successful approach, and real-world deployment. The journey from 50% accuracy (failure) to 74.6% accuracy (success) to a functional user application illustrates that AI development is iterative, requires domain knowledge, and must prioritize user needs alongside technical performance.

The Shot Coach application achieves its goal of democratizing access to shot analysis. With public deployment on Hugging Face Spaces (`https://huggingface.co/spaces/paulgk/badminton-shot`), the tool is now accessible to any recreational player with a smartphone or laptop—no installation required. Acknowledged limitations remain around camera angles and single-shot assumptions. By following AI governance principles—transparency, privacy, user control—the system builds trust while providing actionable feedback.

Most importantly, this project proves that negative results, when analyzed rigorously, are as valuable as positive ones. Knowing that pose-only approaches fundamentally cannot work for this problem saves future researchers from attempting the same path.

Acknowledgments

This project used the ShuttleSet dataset from Wei-Yao Wang et al. (2023), MediaPipe Pose from Google Research, and PyTorch/OpenCV frameworks. Code development, debugging, and report preparation were assisted by Claude Code (Anthropic) and ChatGPT (OpenAI). All model training, evaluation, and application development were performed by the author.

References

[1] Wei-Yao Wang et al. *ShuttleSet: A Human-Annotated Stroke-Level Singles Dataset for Badminton Tactical Analysis*. CoRR, abs/2306.04948, 2023.

[2] Google Research. *MediaPipe Pose*. `https://google.github.io/mediapipe/solutions/pose.html`, 2023.

[3] Kaiming He et al. *Deep Residual Learning for Image Recognition*. CVPR 2016.

[4] Tsung-Yi Lin et al. *Focal Loss for Dense Object Detection*. ICCV 2017.

[5] Streamlit Inc. *Streamlit: The fastest way to build data apps*. `https://streamlit.io`, 2024.

[6] Hugging Face. *Spaces: ML Demo Apps on Hugging Face*. `https://huggingface.co/spaces`, 2024. Shot Coach deployment: `https://huggingface.co/spaces/paulgk/badminton-shot`.

[7] Paul George Karippaparambil. *iti123_v2: Badminton Shot Classification Project*. GitHub, 2026. `https://github.com/paulgk/iti123_v2`.

[8] Anthropic. *Claude Code: AI-Powered Command Line Coding Tool*. `https://www.anthropic.com/claude/code`, 2025.

[9] Adam Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. NeurIPS 2019.